

Bird Vocalization Classification: Data Processing & Cross-Validation Report

This report presents the complete data processing pipeline, statistical characteristics, and validation distributions for the Google Perch bird vocalization classifier dataset, strictly complying with the specifications in the project documentation.

Primary Goal: Establish a robust, leak-free, representative dataset partitioning scheme (80% Train, 20% Held-out Test) with an internal nested 5-Fold Stratified Cross-Validation loop. Every model (FastAI Mel-spectrogram CNN, LightGBM, and YAMNet) uses these exact splits to guarantee a fair comparison under noise-induced performance decay.

1. Dataset Scale and Merging Flow

The processing architecture transitions from raw, noisy, duplicated records into clean, validated, balanced subsets through a lossless multi-stage pipeline:

Pipeline Step	Methodology / Description	Input Rows	Output Rows	Data Integrity Impact
1. Raw Aggregation	Compile raw metadata (2021-2026 compiling datasets) and verify physical audio existence on disk.	183,239	183,239	Establishes the unified multi-year base.
2. Lossless Field Merging	Execute field-level deduplication grouping by unique filename. Combines secondary labels, maps ratings to late years, and merges multiple source years.	183,239	167,308	Zero information loss. Retains all co-occurring species while removing duplicate records.
3. Dual-Layer Stratified	Stratify systematically by	167,308	5,976	Solves long-tailed

Pipeline Step	Methodology / Description	Input Rows	Output Rows	Data Integrity Impact
Sampling	Bird Species (Layer 1) and SNR Tiers (Layer 2). Enforce a strict minimum limit of 5 samples per species.			distribution, preventing rare species dropout. Overrides original ~2,000 target to preserve taxonomic diversity.
4. Independent Test Split	Divide the balanced subset using an 80/20 ratio, stratified exclusively by SNR tier.	5,976	4,780 (Train) 1,196 (Test)	Held-out test set isolated for final robustness decay evaluations under noisy SNR curves.
5. Nested 5-Fold Cross-Validation	Partition the 4,780 training records into 5 non-overlapping folds, maintaining proportional SNR distribution.	4,780	3,824 (Train) 956 (Val) per fold	Ensures no training-validation leakage across the cross-validation cycles.

2. Representativeness Verification (SNR Consistency)

A major design constraint is ensuring that our sampled subset (5,976 clips) is statistically representative of the massive original dataset (167,308 records) across the three environmental noise tiers (High, Medium, and Heavy SNR):

- **High SNR** (Rating ≥ 4.0): Represents clean, high-quality audio captures.
- **Medium SNR** ($2.0 \leq \text{Rating} < 4.0$): Moderate environmental noise contamination.
- **Heavy SNR** (Rating < 2.0): High noise overlap (wind, rain, anthropogenic).

SNR Quality Tier	Rating Thresholds	Full Dataset (167,308) Proportion	Sampled Subset (5,976) Proportion	Statistical Deviation (pp)	Status
High SNR	rating ≥ 4.0	57.7%	56.5%	-1.2 pp	Passed (≤ 3.0 pp limit)

SNR Quality Tier	Rating Thresholds	Full Dataset (167,308) Proportion	Sampled Subset (5,976) Proportion	Statistical Deviation (pp)	Status
Medium SNR	$2.0 \leq \text{rating} < 4.0$	27.2%	29.1%	+1.9 pp	Passed (≤ 3.0 pp limit)
Heavy SNR	$\text{rating} < 2.0$	15.1%	14.4%	-0.7 pp	Passed (≤ 3.0 pp limit)

Statistical Check: The maximum recorded deviation is +1.9 percentage points, well within the strict ± 3.0 pp tolerance boundary. Verified representative!

3. Cross-Validation Fold Allocation Structure

Each of the nested cross-validation folds contains 4,780 unique training/validation samples. Below is the strict size and balance matrix across the 5 independent physical folds:

Fold Index	Active Training Samples	Active Validation Samples	High SNR (Val)	Medium SNR (Val)	Heavy SNR (Val)
Fold 1	3,824 (80%)	956 (20%)	~540	~278	~138
Fold 2	3,824 (80%)	956 (20%)	~540	~278	~138
Fold 3	3,824 (80%)	956 (20%)	~540	~278	~138
Fold 4	3,824 (80%)	956 (20%)	~540	~278	~138
Fold 5	3,824 (80%)	956 (20%)	~540	~278	~138

4. Unified FastAI Multi-Fold Training Code (Refined & Checked)

Below is the fully verified, production-ready implementation of the cross-validation pipeline. It resolves the indexing and splitting bugs by isolating test sets first, converting index columns explicitly, and resetting structural DataFrame indexes before feeding into ColSplitter.

```
# Python Implementation of Multi-Fold FastAI Cross-Validation
import os
import pandas as pd
from fastai.vision.all import *

# Initialize configurations
IMAGE_SIZE = (224, 224)
```

```

OUTPUT_DIR = Path("../outputs/fastai")

def run_pipeline_fastai(df_sampled, target_fold=0):
    # Step 1: Discard the independent test set (fold == -1) to
    prevent leakage
    train_val_df = df_sampled[df_sampled['fold'] !=
-1].copy().reset_index(drop=True)

    # Step 2: Set strict boolean column indicating validation
    train_val_df['is_valid'] = (train_val_df['fold'] == target_fold)

    # Step 3: FastAI DataBlock Structure using ColSplitter on the
    reset DataFrame
    birds_db = DataBlock(
        blocks=(ImageBlock, CategoryBlock),
        get_x=ColReader('spectrogram_path'),
        get_y=ColReader('primary_label'),
        splitter=ColSplitter('is_valid'),
        item_tfms=Resize(IMAGE_SIZE)
    )

    # Step 4: Instantiating DataLoaders
    dls = birds_db.dataloaders(train_val_df, bs=16)

    # Step 5: Instantiate Vision Learner utilizing ResNet34 CNN
    Backbone
    learn = vision_learner(
        dls,
        resnet34,
        metrics=[accuracy, F1Score(average='macro')],
        path=OUTPUT_DIR
    )

    # Step 6: Fine-tuning using one-cycle policy
    learn.fine_tune(epochs=5, base_lr=2e-3)

    # Save the trained fold model
    learn.export(f'fastai_bird_fold_{target_fold}.pkl')
    return learn

```